

Prerequisite Linux Commands

Read this before the lab. Lab 3 (Processes) assumes you can already use every command below — everything here is used in the lab without further explanation. Work through each section in a terminal; reading is not enough.

Each section lists the **base commands** first (what the command does on its own), then the **flags** the lab uses with them (what each flag adds).

Contents

1. Navigating and managing files

2. Building C programs

3. Reading manual pages

4. Processes and `ps`

5. The `/proc` filesystem

6. Redirection, pipes, background jobs
7. Working in two terminals

8. Shared memory and FIFO objects

9. Packaging your submission

10. Other everyday commands

11. Self-check

12. Assignment (Parts A and B)

1. Navigating and managing files

COMMANDS

Command	What it does	Used in lab for
<code>pwd</code>	Print current directory	Knowing where you are
<code>cd</code>	Change directory (<code>~</code> = your home)	Setup
<code>mkdir</code>	Create a directory	Setup
<code>ls</code>	List directory contents	Inspecting <code>/dev/shm</code> , spotting the <code>p</code> on a FIFO
<code>cp</code>	Copy files	Copying starter code
<code>rm</code>	Delete a file (no trash, no undo)	Removing test FIFOs
<code>cat</code>	Print a file's contents	Reading <code>/proc</code> entries
<code>head</code>	Print the first 10 lines of a file	Trimming long <code>/proc</code> output

FLAGS

Flag	Command	What it adds
<code>-p</code>	<code>mkdir</code>	Creates parent directories as needed; never errors if the directory exists
<code>-l</code>	<code>ls</code>	Long listing: permissions, owner, size, and the file-type character
<code>-r</code>	<code>cp</code>	Recurse into directories (required to copy a folder)
<code>-20</code>	<code>head</code>	Show the first 20 lines instead of the default 10
<code>-c 60</code>	<code>head</code>	Show the first 60 bytes instead of lines (used on the shared-memory segment)

Try it now:

```
mkdir -p ~/oslab/lab3 && cd ~/oslab/lab3 && pwd
```

The `&&` chains commands: the next one runs only if the previous succeeded.

2. Building C programs

COMMANDS

Command	What it does
<code>gcc</code>	Compile a C source file into an executable
<code>make</code>	Run the <code>Makefile</code> in the current directory — builds everything
<code>make clean</code>	Remove built binaries (and, in this lab, leftover shared memory segments)
<code>./program</code>	Run an executable from the current directory — the <code>./</code> is mandatory

FLAGS

Flag	Command	What it adds
<code>--version</code>	<code>gcc</code>	Print the compiler version (used to verify your toolchain)
<code>-o name</code>	<code>gcc</code>	Name the output executable (<code>a.out</code> otherwise)
<code>-lrt</code>	<code>gcc</code>	Link the POSIX realtime library — required for <code>shm_open</code> in Part 5 and the assignment

Putting them together:

```
gcc --version
gcc -o fork_basic fork_basic.c
gcc -o shm_producer shm_producer.c -lrt
./fork_basic
```

Why `./` ? The current directory is not on your `PATH`, so `fork_basic` alone gives `command not found`.

3. Reading manual pages *(you will need this constantly)*

COMMAND

Command	What it does
<code>man</code>	Open the manual page for a command, system call, or library function

SECTION NUMBERS

`man` takes no flag here; the number between `man` and the name selects the manual *section*:

Invocation	Section	Covers
<code>man 1 ls</code>	1	Shell commands
<code>man 2 fork</code>	2	System calls
<code>man 3 printf</code>	3	C library functions

The section matters: `man fork` might land on the wrong page; `man 2 fork` is explicit.

Inside `man`: `Space` scrolls, `/word` searches, `n` next match, `q` quits.

If `man 2 fork` fails on Ubuntu/WSL2:

```
sudo apt install manpages-dev
```

During the lab you are expected to answer return-value and error questions **from the man pages**, not from a search engine.

4. Processes and `ps`

COMMANDS

Command	What it does
<code>ps</code>	List your processes in this terminal
<code>ps tree</code>	Show the whole process tree — makes parent/child relationships visible
<code>top</code>	Live process view (quit with <code>q</code>)
<code>kill</code>	Send a signal to a process (polite termination by default)

FLAGS

Flag	Command	What it adds
<code>-e</code>	<code>ps</code>	Every process on the system, not just yours
<code>-l</code>	<code>ps</code>	Long format — adds the PCB columns used in Part 0: <code>S</code> , <code>PID</code> , <code>PPID</code> , <code>PRI</code> , <code>NI</code> , <code>SZ</code> , <code>WCHAN</code>
<code>aux</code>	<code>ps</code>	BSD-style: <code>a</code> = all users, <code>u</code> = user-oriented columns, <code>x</code> = include processes with no terminal. The <code>STAT</code> column shows <code>Z</code> for zombies (Part 3)
<code>-p 1234</code>	<code>ps</code>	Only the process with PID 1234
<code>-p</code>	<code>ps tree</code>	Show PIDs next to each process name
<code>-9</code>	<code>kill</code>	Send SIGKILL (forced, cannot be caught) — for stuck experiments only

The two `ps` variants the lab uses:

```
ps -el      # Part 0: PCB columns
ps aux      # Part 3: finding zombies
```

Two shell variables you will use directly:

```
echo $$     # PID of the current shell
echo $!     # PID of the most recent background job
```

Killing processes:

```
kill 1234    # polite termination (SIGTERM)
kill -9 1234 # forced kill (SIGKILL)
```

5. The `/proc` filesystem

`/proc` is a fake filesystem the kernel generates on demand — every process's PCB is exposed under `/proc/<PID>/`. Part 0 is built entirely on this. No new commands or flags here — just `cat`, `ls`, and `head` from Section 1 pointed at kernel-generated files:

```
cat /proc/$$/status | head -20    # human-readable PCB: state, PIDs, memory
cat /proc/$$/stat                 # same data, one machine-readable line
ls /proc/$$/fd                   # open file descriptors (0=stdin, 1=stdout, 2=stderr)
cat /proc/$$/maps | head         # memory layout: text, heap, stack
cat /proc/1/status                # PID 1 – init/systemd, ancestor of everything
```

Note `$$` expands to your shell's PID before the command runs, so `cat /proc/$$/status` means "show my shell's own status".

6. Redirection, pipes, and background jobs

These are shell *operators*, not commands, but you cannot complete the lab without them: the buffering trap (Part 1), the zombie observation (Part 3), and the FIFO demonstration (Part 6) all rely on them.

OPERATORS

Operator	What it does
<code>></code>	Redirect stdout to a file (this changes buffering — the Part 1 trap)
<code> </code>	Pipe: stdout of the left command becomes stdin of the right (an <i>ordinary pipe</i> , discussed in Q6.2)
<code>&</code>	Run the command in the background, giving your prompt back

SUPPORTING COMMANDS

Command	What it does
<code>grep pattern</code>	Keep only lines containing the pattern
<code>awk '\$8 ~ /Z/'</code>	Keep only lines whose 8th column matches <code>Z</code> — needed because the state column position varies
<code>jobs</code>	List background jobs
<code>fg</code>	Bring the most recent background job to the foreground

In use:

```
./fork_basic > out.txt    # Part 1: the buffering trap
cat out.txt

ls -l | grep txt          # ordinary pipe between two commands
ps aux | awk '$8 ~ /Z/'   # Part 3: how you find zombies

./zombie &               # run in the background
jobs
fg
```

7. Working in two terminals

Parts 5, 6, and the graded assignment run two cooperating programs **in separate terminals** (producer/consumer, reader/writer). Before the lab, make sure you can:

- Open a second terminal window or tab (in VS Code: `Ctrl` + `Shift` + ```; on WSL2: open a second Ubuntu window).
- `cd` both terminals to the same directory.
- Understand that a program printing nothing may be **blocked, not hung** — `fifo_writer` sits inside `open()` until a reader attaches. `Ctrl` + `C` interrupts a foreground program if you really do need to abort it.

8. Shared memory and FIFO objects on disk

Both IPC mechanisms in this lab leave visible objects in the filesystem. Know how to find and remove them, or leftovers from a crashed run will corrupt your next run.

COMMANDS

Command	What it does
<code>mkfifo</code>	Create a named pipe (FIFO) in the filesystem
<code>ls -l /dev/shm/</code>	List POSIX shared memory segments — they live here as files
<code>rm</code>	Remove a FIFO or a leaked shared memory segment, like any file

In use:

```
ls -l /dev/shm/           # what segments exist right now?
rm /dev/shm/os_lab_shm    # manual cleanup if shm_unlink was never called

mkfifo /tmp/demo_fifo     # create a named pipe
ls -l /tmp/demo_fifo      # first character is 'p' — it's a pipe, not a regular file
echo "hello" > /tmp/demo_fifo & # write to it in the background (blocks until a reader appears)
cat /tmp/demo_fifo        # read it
rm /tmp/demo_fifo
```

Shared memory segments **persist until reboot** unless unlinked — check `/dev/shm/` whenever a shared-memory program misbehaves.

9. Packaging your submission

COMMAND

Command	What it does
<code>tar</code>	Bundle files into a single archive (and back)

FLAGS

Flag	What it adds
c	Create an archive
z	Compress with gzip
f name	Use this archive filename (must come last before the name)
t	List an archive's contents without extracting
x	Extract an archive

In use:

```
tar czf lastname_lab3.tar.gz mysh.c bounded_buffer.c report.md screenshots/
tar tzf lastname_lab3.tar.gz      # verify contents before submitting
```

10. Other everyday commands you should know

The lab manual does not use these directly, but you will need them constantly while working, and your lab instructor will assume you know them.

FILES AND TEXT

Command	What it does
touch file	Create an empty file (or update its timestamp)
mv old new	Move or rename a file
echo "text"	Print text — also used to write into files with >
less file	Page through a file (Space scrolls, / searches, q quits — same keys as man)
tail file	Print the last 10 lines of a file
wc file	Count lines, words, and bytes
sort file	Sort lines alphabetically
find dir -name "*.c"	Search a directory tree for files by name
which program	Show which executable the shell would run — the PATH lookup made visible
file name	Report what a file actually is (C source, executable, FIFO, ...)
chmod +x script	Make a file executable
nano file	Simple terminal text editor (or vim if you know it)

SYSTEM INFORMATION

Command	What it does
<code>whoami</code>	Your username
<code>uname</code>	Kernel name — useful to confirm you're on Linux, not macOS
<code>uptime</code>	How long the system has been running, plus load average
<code>df</code>	Disk space per filesystem
<code>du</code>	Disk usage of a directory
<code>free</code>	Memory usage
<code>date</code>	Current date and time
<code>history</code>	Your recent commands — re-run one with <code>!<number></code>
<code>clear</code>	Clear the terminal (or <code>Ctrl+L</code>)

MISCELLANEOUS

Command	What it does
<code>sleep 30</code>	Do nothing for 30 seconds — the standard test subject for background jobs and <code>kill</code>
<code>time ./program</code>	Run a program and report how long it took
<code>sudo command</code>	Run a command as administrator (needed for <code>apt install</code>)
<code>apt install pkg</code>	Install software on Ubuntu/WSL2 (always with <code>sudo</code>)

FLAGS

Flag	Command	What it adds
<code>-a</code>	<code>uname</code>	All system info: kernel version, architecture
<code>-h</code>	<code>df</code> , <code>du</code> , <code>free</code>	Human-readable sizes (<code>1.2G</code> instead of bytes)
<code>-s</code>	<code>du</code>	Summary: one total for the directory instead of every subdirectory
<code>-l</code>	<code>wc</code>	Count lines only
<code>-f</code>	<code>tail</code>	Follow: keep printing as the file grows (<code>Ctrl+C</code> to stop)
<code>-n</code>	<code>sort</code>	Sort numerically instead of alphabetically
<code>-name</code>	<code>find</code>	Match files by name pattern
<code>+x</code>	<code>chmod</code>	Add execute permission

In use:

```
which gcc                # where does the compiler actually live?
find . -name "*.c" | wc -l  # how many C files in this lab?
file fork_basic          # confirm it's an executable, not source
du -sh ~/oslab/lab3      # how big is my lab directory?
history | grep gcc        # what compile commands have I run?
```

Self-check *(5 minutes)*

You are ready for the lab if you can do all of these without looking anything up:

1. Create `~/oslab/test`, `cd` into it, and confirm with `pwd`.
2. Compile a hello-world C file with `gcc` and run it with `./`.
3. Open `man 2 wait`, search inside it for `WIFEXITED`, and quit.
4. Print your shell's PID two different ways (`echo $$` and `ps`).
5. Show your shell's open file descriptors from `/proc`.
6. Run `sleep 30 &`, find its PID with `ps` and `$!`, and kill it.
7. Redirect `ls -l` into a file, then filter that file for a word with `grep`.
8. List `/dev/shm/` and explain what would appear there during Part 5.

Assignment

The assignment has two parts. **Part A** exercises the core commands (Sections 1–6, 10); **Part B** exercises the IPC-related commands — shared memory and FIFOs (Sections 7–8). Complete the tasks **in order** in a fresh terminal. Where a task says *capture*, redirect the output into the named `.txt` file — that's Section 6 practice in itself. Do all work inside one directory:

```
mkdir -p ~/oslab/prereq && cd ~/oslab/prereq
```

Part A — Core commands

Task A1 — Files and directories (Sections 1, 10)

1. Inside `~/oslab/prereq`, create the directory tree `work/src` and `work/backup` with a single command.
2. Create an empty file `work/src/notes.txt` with `touch`, then rename it to `work/src/journal.txt` with `mv`.
3. Write the line `prerequisites practice` into it using `echo` and `>`.
4. Copy the entire `work/` tree to `work2/` with one `cp` command.
5. Capture proof: `ls -lR work work2 > taskA1.txt`

Task A2 — Compile and run (Section 2)

1. Create `hello.c` (any editor) that prints your student ID.
2. Compile it to an executable named `hello` using the `-o` flag and run it with `./`.
3. Capture: `gcc --version > taskA2.txt`, then append your program's output with `./hello >> taskA2.txt`. Note that `>>` appends while `>` overwrites — if the file ends up containing only your program's output, you used `>` where `>>` was needed.

Task A3 — Man pages (Section 3)

Answer in `taskA3.txt` (plain text, one answer per line), using **only** man pages:

1. According to `man 2 fork`, what does `fork()` return in the child process?
2. According to `man 2 wait`, what does the macro `WIFSIGNALED(status)` tell you?
3. `man 1 kill` and `man 2 kill` document different things. What is each one?

Task A4 — Processes and `/proc` (Sections 4, 5)

1. Print your shell's PID with `echo $$`, then confirm it with `ps -p`.
2. Find your shell's **parent** PID in `/proc/$$/status`, then identify what that parent process is with `ps -p`.
3. From `/proc/$$/status`, record these fields: `State`, `Pid`, `PPid`, `VmSize`, `Threads`.
4. Capture all of the above: run the commands again with `>> taskA4.txt` after each.

Task A5 — Pipes, redirection, background jobs (Section 6)

1. Count how many processes are running on your system using `ps -e` piped into `wc -l`. Capture into `taskA5.txt`.
2. Run `sleep 300 &`. Record its PID using `$!` (append to `taskA5.txt`).
3. Find the same sleep process with `ps aux` piped through `grep` (append the matching line).
4. Kill it, then show it is gone with the same `ps | grep` pipeline (append).

Task A6 — Investigation one-liners (Section 10)

Build each answer as a **single pipeline** and capture all three into `taskA6.txt`:

1. How many `.c` files does the Lab 3 starter code contain? (`find` + `wc`)

- 2. Where is the `gcc` executable on your system, and what kind of file is your compiled `hello`? (`which` , `file`)
- 3. How many commands from your history so far involved `gcc`? (`history` + `grep` + `wc`)

Part B — IPC: shared memory and FIFOs

Task B1 — `/dev/shm` as a filesystem (Section 8)

Shared memory segments appear in `/dev/shm` as ordinary-looking files. Prove it without writing any C:

- 1. Capture the current contents: `ls -l /dev/shm/ > taskB1.txt`
- 2. Create a fake "segment": `echo "not really shared memory" > /dev/shm/prereq_test`
- 3. Append proof it appeared (`ls -l /dev/shm/ >> taskB1.txt`) and read it back with `cat` (append that too).
- 4. Remove it with `rm` and append a final listing showing it gone.
- 5. In one appended sentence: after Part 5 of the lab, what would a leftover file here mean, and which call should have removed it?

Task B2 — Ordinary pipe vs named pipe (Sections 6, 8)

- 1. Run `ps aux | grep $$` — an **ordinary pipe** between two processes. Capture the output into `taskB2.txt`.
- 2. Create a **named pipe** at `/tmp/prereq_fifo` with `mkfifo`.
- 3. Append proof it is a pipe: `ls -l /tmp/prereq_fifo >> taskB2.txt` (the first character must be `p`).
- 4. In one appended sentence: the ordinary pipe in step 1 existed only for that command; what can the named pipe do that it couldn't? (Hint: this is exactly the Q6.2 distinction in the lab manual.)

Task B3 — Two terminals through the FIFO (Sections 7, 8)

- 1. Open a **second terminal** and `cd` it to `~/oslab/prereq`.
- 2. In terminal 1 run `cat /tmp/prereq_fifo`; in terminal 2 send your name through it with `echo` and `>`. Confirm it appears in terminal 1.
- 3. Which terminal blocked, and until when? Answer in one sentence in `taskB3.txt`.
- 4. Now do it in the **other order**: run the `echo` (with `&`) before any reader exists, then attach the `cat`. What happened to the `echo` in the meantime? Append one sentence.
- 5. Remove the FIFO.

Wrap-up — package both parts (Section 9)

- 1. From `~/oslab/prereq`, create `lastname_prereq.tar.gz` containing `hello.c`, `hello`, and all nine `task*.txt` files.
- 2. Verify the archive by **listing its contents without extracting it**, and confirm that every file is present.

What to submit

Item	Shows you can
<code>taskA1.txt</code> – <code>taskA6.txt</code>	Use every core command family in this document
<code>taskB1.txt</code> – <code>taskB3.txt</code>	Inspect and clean up the IPC objects Lab 3 creates
<code>hello.c</code> + <code>hello</code>	Compile and run C from the terminal
<code>lastname_prereq.tar.gz</code>	Package a submission — the same format Lab 3 requires

Submit the single archive. If any task took you more than a few minutes, reread that section before the lab — these same commands appear there under time pressure.